

ENME441

Brokered Communication

MQTT + HiveMQ and user interfacing

(MQTT = Message Query Telemetry Transport)

Topics



Brokered comms via MQTT

- Why not peer-to-peer?
- MQTT publish / subscribe model
- HiveMQ broker
- hivemq.py on the ESP32



User interfacing

- Interface options for brokered comms
- MQTT through a Web browser using WebSocket
- HTML + Javascript template

01

Brokered comms via MQTT



Bidirectional communications through a cloud MQTT broker

umd-iot Isolates Everything

Your ESP32 can reach out to internet servers, but nothing can contact you from the outside since all inbound connections are blocked.

1

Cross-subnet isolation

Devices on umd-iot cannot talk to devices on eduroam, and vice versa. Your laptop cannot reach your ESP32 directly.

2

Peer-to-peer isolation

Two devices on umd-iot cannot reach each other either, even sitting side by side on the same network.

3

All inbound connections blocked

You cannot SSH in, send it an HTTP request, or open a socket from outside.

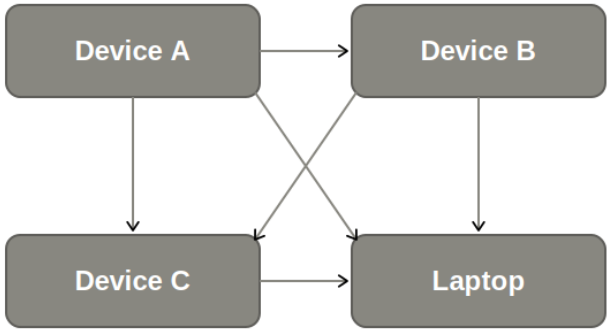
4

Outbound connections are allowed

The ESP32 can open a bidirectional connection to an outside device or server

Peer-to-Peer vs. Brokered Communications

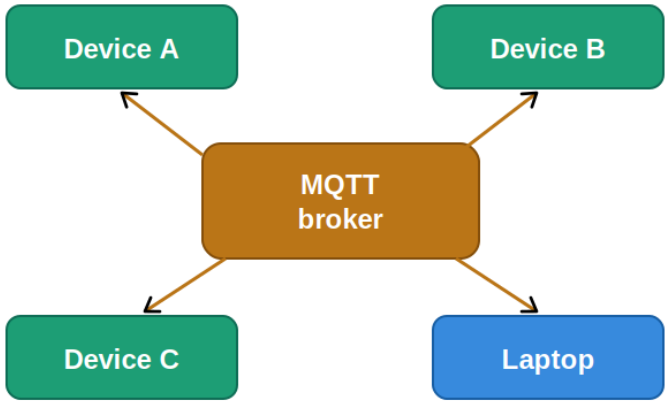
Peer-to-peer (problematic)



Issues:

- Every device must know every other
- N devices = N×(N-1) connections
- Requires open inbound ports
- Blocked by NAT and firewalls
- No message persistence

Broker-based pub/sub (scalable)



Benefits:

- Devices only connect to one endpoint
- N devices = N connections
- Only outbound connections needed
- Works through NAT and firewalls
- Broker stores messages if device offline

How publish/subscribe works

Publish:	A device sends a message to a named topic (e.g., "course/student42/temperature")
Subscribe:	Any client registers interest in a topic; the broker delivers matching messages
Decouple:	Publishers and subscribers don't know about each other — only the broker does
Persist:	The broker can hold retained messages so new subscribers get the last known value

MQTT: Lightweight Pub/Sub Protocol for IoT

Message Query Telemetry Transport allows a broker to manage bidirectional communication between publishers and subscribers.

1

Publish / subscribe to selected “topics”

ESP32 publishes data to a topic, and a laptop subscribes to the same topic to get messages (and visa versa). Because the ESP32 initiates the communication, messages can be returned to the ESP32 by the broker, even on networks that block inbound connections.

2

Built for constrained devices

Message headers as small as 2 bytes, keep-alive and reconnect built in, client library with small memory footprint.

3

Retained messages

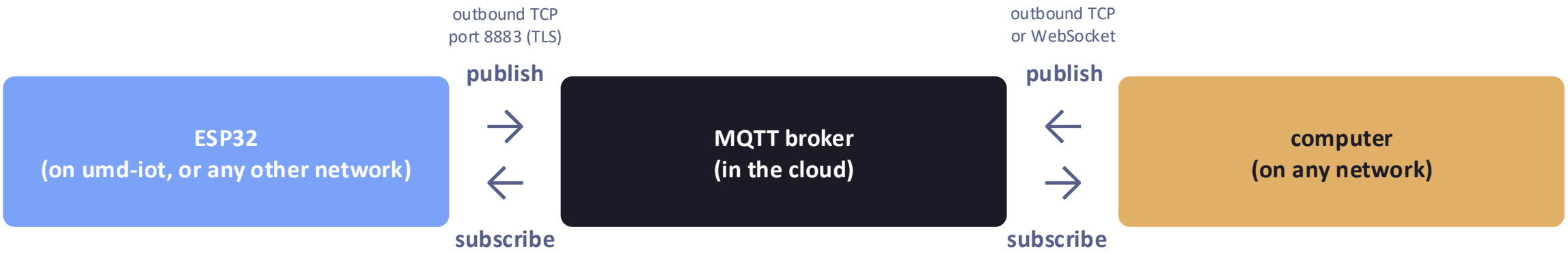
Retained message lets a device that just reconnected immediately learn the current state.

4

Last Will and Testament

You register an 'offline' message at connect time and the broker publishes it for you if your device drops unexpectedly.

One-Way Connection, Bi-Directional Messaging



- **Both sides connect with the MQTT broker**

  Neither device ever accepts an incoming connection, so no firewall or NAT rules are broken.

- **TCP is bidirectional**

  Once an MQTT connection is open, the broker can push messages to connected nodes at any time.

- **Hold connection open with `keep-alive`**

  The MQTT protocol pings periodically so the connection survives indefinitely.

HiveMQ Cloud-Based MQTT Broker

1

A broker is just a program that handles MQTT communications

MQTT defines the message format. The broker is a program that speaks the MQTT protocol, sits at a fixed address, holds every client's connection open, and routes messages in both directions.

2

The broker must live where both sides can reach it

You can run your own MQTT server (e.g. Mosquitto) on a laptop, but this puts the broker behind the same firewall and NAT that blocked you in the first place. A broker on the public internet is reachable from umd-iot, eduroam, or any other internet-connected network.

3

HiveMQ Cloud offers a robust free tier

HiveMQ provides a public hostname, a valid TLS certificate, always-on uptime, and flexible MQTT messaging.

4

Any MQTT client can connect to HiveMQ

umqtt.simple on the ESP32, MQTT.js in a browser, MQTT Explorer on your desktop, paho-mqtt in a Python script, etc.

HiveMQ Setup



Follow the instructions in brokered_comms_setup.pdf to prepare your ESP32 and laptop for working with MQTT through the HiveMQ broker.
https://github.com/ddevoe-umd/enme441/blob/main/other_content/brokered_comms_setup.pdf

Key steps:

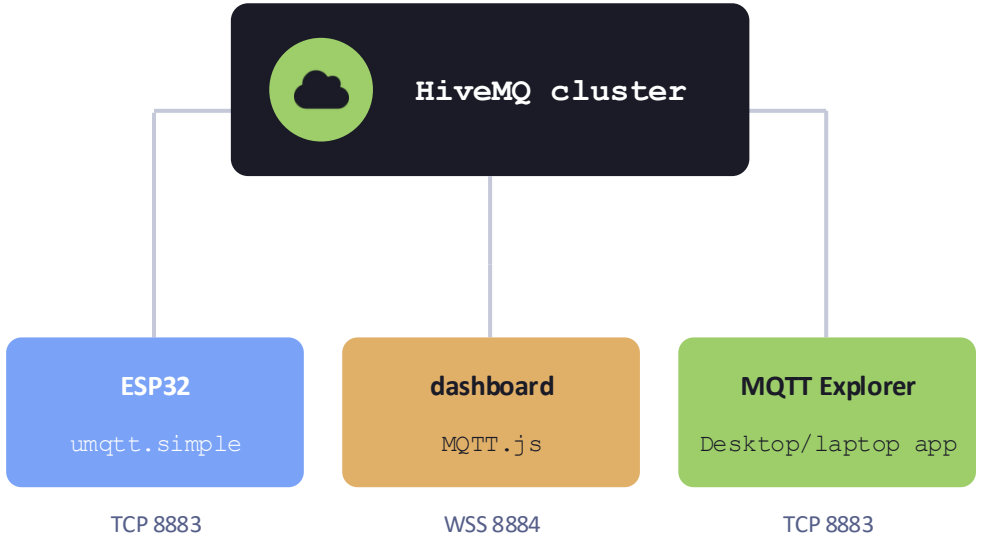
- Install the MQTT library on your ESP32:
```
import mip
mip.install('umqtt.simple')
```
- Install the hivemq.py example code on your ESP32:
```
mip.install('github:ddevoe-umd/enme441/micropython/10_Brokered_Communication/hivemq.py')
```
- Install MQTT Explorer on your laptop:
  https://mqtt-explorer.com

Shared HiveMQ Cluster for ENME441

All class devices will connect to a single HiveMQ cluster with one shared set of credentials.

CONNECTION DETAILS

HOST	69702dc0158e4b3ea7e406ed4c78ae13.s1.eu.hivemq.cloud
TCP PORT	8883 SSL/TLS · umqtt.simple on the ESP32
WEBSOCKET (WSS) PORT	8884 SSL/TLS · MQTT.js in the browser
USERNAME	enme441
PASSWORD	enme441iot
TLS	required the cluster will refuse plaintext connections



- **Key constants defined in config.py**

  hivemq.py reads MQTT_BROKER, MQTT_PORT, MQTT_USER, and MQTT_PASS from config.py.

- **Shared credentials**

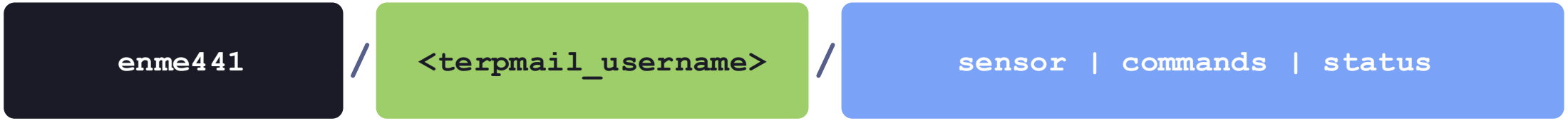
  The broker cannot tell one user from another since every client authenticates as enme441.

- **Nothing is private**

  Anyone can subscribe to enme441/# and watch the data traffic. The use of user-specific topic names keeps the data streams untangled, but not private!

ENME441 Broker Topics

Hierarchical strings separated by forward slashes. The string structure is arbitrary and can be redefined as needed for different applications.



Shared prefix *User-specific namespace* *message type*

`enme441/<terpmail>/sensor`	*ESP32 → broker → laptop*	Sensor and state data, published as JSON every few seconds.
`enme441/<terpmail>/commands`	*laptop → broker → ESP32*	Control commands. The ESP32 subscribes here and acts in on_message().
`enme441/<terpmail>/status`	*ESP32 → broker (retained)*	online / offline. Set as the Last Will so the broker publishes it if you drop.

hivemq.py — Connecting to the Broker

`start_comms()` builds the client, connects over TLS, and subscribes

```
from umqtt.simple import MQTTClient

def start_comms():
    ssl_params = {'server_hostname': config.MQTT_BROKER}
    client = MQTTClient(
        client_id = config.TERPMAIL_USERNAME,
        server    = config.MQTT_BROKER,
        port      = config.MQTT_PORT,
        user      = config.MQTT_USER,
        password  = config.MQTT_PASS,
        ssl       = True,
        ssl_params = ssl_params
    )

    client.set_callback(on_message)    # set up callback for check_msg()
    client.set_last_will(config.TOPIC_STATUS, b'offline', retain=True)
    client.connect()
    client.subscribe(config.TOPIC_COMMANDS)
    client.publish(config.TOPIC_STATUS, b'online', retain=True)
    return client
```

- **Uses Transport Layer Security (TLS)**

  TLS is a security protocol that encrypts data sent between two systems over a network to keep it private (https vs. http).

- **set_callback()**

  Define the callback function to trigger when checking for broker messages via client.check_msg()
  – see next slide.

- **set_last_will(...)**

  Registered with the broker before connect. If the board drops off, the broker publishes 'offline' on your behalf.

- **Send raw bytes only**

  MQTT is a binary protocol, so published payload is just a sequence of bytes (not strings). Use `b'my_string'` to convert strings to bytes.

- **subscribe before you publish**

  Do it inside start_comms() so a reconnect always re-establishes the subscription. You can publish to a topic as many times as you like once subscribed.

hivemq.py — Receiving and Publishing

`on_message()` callback for check_msg() to handle all incoming messages
`run()` keeps the connection alive while looking for incoming messages & sending outgoing messages

```
led = Pin(8, Pin.OUT)           # built-in LED

def on_message(topic, msg):
    data = json.loads(msg)
    cmd = data.get('cmd', '')
    if cmd == 'LED_ON':
        led.value(0)            # inverted (pull-up on LED)
    elif cmd == 'LED_OFF':
        led.value(1)

def run():
    client = start_comms()
    while True:
        client.check_msg()      # non-blocking

        payload = json.dumps({'sensor_1': v1,
                              'sensor_2': v2,
                              'uptime': time.ticks_ms()})
        client.publish(config.TOPIC_SENSOR, payload.encode())

        time.sleep(5)           # << keep-alive interval
```

- **The callback fires from check_msg()**

  It is not a thread or an interrupt — nothing is delivered from the server until you call check_msg().

- **Extend on_message(), not the loop**

  This is where every new piece of hardware in your project gets its command.

- **Payload is always in JSON format**

  Use json.dumps() to package output to the broker and json.loads() to process incoming messages from the broker.

- **Encode payload as a bytes object**

  payload.encode(), or alternately bytes(payload, 'utf-8')

- **sleep(5)**

  Never block more than ~30 s without calling check_msg() or ping(), or the broker treats the client as dead and disconnects it.

Testing from Your Laptop

Use MQTT Explorer (mqtt-explorer.com) as an easy way to interact with the broker from your laptop.

Host	`69702dc0158e4b3ea7e406ed4c78ae13.s1.eu.hivemq.cloud`
Port	`8883`
Username / Password	`enme441 / enme441iot (shared course credentials)`
Encryption (TLS)	`enabled — required`

Publish a command from the Publish panel:

```
topic:   enme441/<terpmail_username>/commands
payload: {"cmd": "LED_ON"}
```

- **Expand the topic tree**

  You should see enme441 > your username > sensor updating every 5 seconds.

- **Check the status**

  It should read 'online'. Unplug the board and it flips to 'offline' — that's the Last Will firing.

- **Try sending LED on/off commands**

  Payload must be JSON format, using double-quotes for strings

MQTT: Things to Watch For

Connection drops after 60–90 seconds

Your loop blocks too long between check_msg() calls. Shorten the sleep, or pass a keepalive argument (the sample sets none).

MQTTException / connection refused

Check port 8883 (not 1883), verify the broker username and password, and confirm ssl=True is set.

ECONNABORTED on first connect

The broker can go dormant after a quiet period. Simply retry the connection.

ESP32 publishes, nothing arrives

Topics are case-sensitive and must match character for character on both ends. Confirm in MQTT Explorer.

Command sent, board ignores it

check_msg() is not being called, or on_message() is raising while parsing. Watch the Thonny REPL — every print() lands there.

Garbled messages or TypeError

Make sure all strings are converted to byte objects before sending to the broker. Note that json.dumps() and json.loads() will automatically convert between strings and bytes.

02

User interfacing

Interacting with the ESP32 (through through the broker) from your laptop



Interface Options

OPTION A

HTML + JavaScript

- MQTT.js over WebSocket
- Runs in any browser
- No backend at all, no file or data access (sandboxed browser)
- **Excellent UI tools and easy to share, but more complex and limited functionality.**

OPTION B

Python desktop app

- paho-mqtt + tkinter
- Python as a shared language with ESP32
- Access to files, numpy, matplotlib, etc.
- **Easy to code, excellent back-end functionality, but limited UI tools and harder to share**

OPTION C

Python + HTML/js frontend

- Flask / FastAPI backend
- Credentials stay server-side
- Most moving parts
- **Best of both worlds but highest complexity**

We will use Option A *(middle ground on complexity, functionality, and UI design)*

Browser Uses WebSocket, not TCP

Browsers cannot open raw TCP sockets, so HiveMQ exposes a WebSocket endpoint alongside TCP.



1. **The broker does not care how you connected**

   Messages published over TCP are delivered over WS and vice versa. Pub/sub routing is identical.

2. **Both endpoints use SSL/TLS and the same credentials**

   Only the transport and the port number differ.

3. **The /mqtt path is required**

   MQTT-over-WebSocket specifies it. Omit it and the connection silently fails.

4. **No backend server needed**

   The broker IS the server. Your page is a static HTML file — open it from your local disk or host it on a web server.

MQTT in a Browser Using Javascript

`MQTT.js` is a JavaScript/Node.js client library for the MQTT protocol. It is the JS equivalent of the umqtt.simple library in MicroPython that can be loaded from an online Content Delivery Network (CDN) so that no Node Package Manager (npm), bundler, or build step is needed

`MQTT.js`		`umqtt.simple`
in the browser · wss://…:8884/mqtt		on the ESP32 · tls://…:8883
`mqtt.connect(url, {username, password})` doConnect() — url = `wss://${broker}:${port}/mqtt`	**connect** TLS on both sides	`MQTTClient(…, ssl=True); client.connect()` start_comms() — host, port, user, pass from config.py
`client.subscribe([`…/sensor`, `…/status`])` inside client.on('connect', …), once the socket is up	**subscribe** each side listens for the other	`client.subscribe(config.TOPIC_COMMANDS)` in start_comms(), immediately after connect()
`client.on('message', (topic, msg) => {…})` event-driven — the browser fires it for you	**callback** trigger action on message receipt	`client.set_callback(on_message)` polled — run() must keep calling client.check_msg()
`data = JSON.parse(payload) → updateSensor()` one card per JSON key; unknown keys create a new card	**…/sensor** ← board publishes	`client.publish(TOPIC_SENSOR, json.dumps({…}))` in run(), once every 5 s alongside check_msg()
`client.publish(topic, JSON.stringify({cmd}))` sendCmd('LED_ON') and sendCustom() button handlers	**…/commands** browser publishes →	`data = json.loads(msg); data.get('cmd')` on_message() → led.value(0) / led.value(1)
`updateSensor('status', payload)` status card; client.end(true) in doDisconnect()	**…/status** ← board publishes, retained	`client.set_last_will(TOPIC_STATUS, b'offline')` publishes b'online' on connect; broker sends offline if you drop

One asymmetry: *MQTT.js is event-driven, so on('message') fires on its own — no check_msg() needed.*

MQTT.js: Connecting from the Browser

```
<script src="https://unpkg.com/mqtt/dist/mqtt.min.js"></script>

const url = `wss://${brokerUrl}:${port}/mqtt`;

const client = mqtt.connect(url, {
  username: 'enme441',
  password: 'enme441iot',
  clientId: `web_${terpmail}_${Math.random()
              .toString(16).slice(2, 8)}`,
  protocolVersion: 4,
  clean: true,
  reconnectPeriod: 5000,
});
```

- **Random clientId suffix**

  Two tabs would otherwise connect with the same ID and the broker would kick the first one off.

- **reconnectPeriod: 5000 ms**

  Sets MQTT.js to reestablish the connection every 5 s.

- **Warning: credentials are in the page source**

  Ok for ENME441, but in production you would proxy through a backend to hide credentials.

MQTT.js: Subscribing and Receiving Messages

MQTT.js is event-driven using registered callbacks.

```
client.on('connect', () => {
  client.subscribe([
    `enme441/${terpmail}/sensor`,
    `enme441/${terpmail}/status`
  ]);
});

client.on('message', (topic, message) => {
  const payload = message.toString();

  if (topic.endsWith('/sensor')) {
    const data = JSON.parse(payload);
    updateSensor('sensor_1', data.sensor_1);
    updateSensor('uptime',   data.uptime);
  }

  if (topic.endsWith('/status')) {
    document.getElementById('val-status')
            .textContent = payload;
  }
});
```

- **Subscribe inside on('connect')**

  Guarantees the connection exists first, and re-subscribes automatically after any reconnect.

- **The payload is a Buffer**

  Call toString() before you do anything with it, then JSON.parse() if it's JSON.

- **Route on the topic**

  endsWith('/sensor') tells you what kind of message arrived without parsing the whole topic string.

- **Same JSON both ways**

  Whatever keys hivemq.py put in json.dumps() are exactly the keys you read here.

MQTT.js: Publishing Commands Back

```
function sendCmd(cmd) {
  const topic = `enme441/${terpmail}/commands`;
  const payload = JSON.stringify({ cmd: cmd });
  client.publish(topic, payload);
}
```

```
<button onclick="sendCmd('LED_ON')">LED On</button>
<button onclick="sendCmd('LED_OFF')">LED Off</button>
```

- **publish(topic, payload)**

  That is the entire API. The broker delivers it to every subscriber of that topic.

- **JSON.stringify ↔ json.loads**

  Both ends agree on JSON, so adding a field means adding a key, with no protocol to redesign.

- **Can send a command & parameters**

  For example: { cmd: 'SERVO', angle: 90 } carries data alongside the command. Handle it in the same callback on the ESP32.

Remember: on the ESP32, we are already waiting for any messages:

```
def on_message(topic, msg):
    data = json.loads(msg)
    if data.get('cmd') == 'LED_ON':
        led.value(0)
```

Interface Template: mqtt_dashboard.html

One HTML file with embedded Javascript, no dependencies beyond MQTT.js, opens in any browser with Javascript enabled:
https://github.com/ddevoe-umd/enme441/blob/main/other_content/mqtt_dashboard.html

1 **Connection panel**

Broker, port 8884, username and password are pre-filled for the course. Type your terpmail username and click Connect — the indicator turns green.

2 **Live sensor cards**

Updates as messages arrive, with no page refresh. A new card will be created automatically for any JSON key the template doesn't recognize.

3 **Command controls**

LED On / LED Off buttons, plus a free-text JSON field for sending anything else to your board.

4 **Scrolling message log**

Every received message with a timestamp. Pair it with the browser console (F12 / Cmd-Opt-I) for debugging when something isn’t working.

Extending the Template

Use the template as a foundation to build on for your course project. Here are some examples of how you might extend the code.

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

// in the message handler:
if (topic.endsWith('/sensor')) {
  const data = JSON.parse(payload);
  readings.push({ time: new Date(), value: data.sensor_1 });
  if (readings.length > 60) readings.shift();
  updateChart(readings);
}
```

- **Add a chart**

  Keep a rolling array of the last N readings and redraw on each message. Chart.js loads from a CDN like MQTT.js does.

```
<input type="range" min="0" max="180" id="servo">
<button onclick="sendServo()">Set Angle</button>

function sendServo() {
  const angle = document.getElementById('servo').value;
  client.publish(`enme441/${terpmail}/commands`,
    JSON.stringify({ cmd: 'SERVO', angle: Number(angle) }));
}
```

- **Add a control**

  New button or slider → sendCmd() with a new command string → a new branch in on_message() on the ESP32.

- **Extend further using a framework**, e.g. React or Vue — npm install mqtt and import it. Only the rendering layer changes. Don't try this unless you already know how to use the framework!

Key Points to Remember

Outbound connections only, both sides

umd-iot blocks every inbound connection. The ESP32 and the browser each dial out to HiveMQ and neither one ever accepts a connection.

Topics are the only contract

enme441/<terpmail>/sensor, /commands, /status. The board and the browser never see each other — they only agree on these strings.

ESP32 + laptop don’t know about each other

The broker and a topic string at the entire interface

Polled vs. event-driven

On the ESP32, run() must keep calling check_msg() or the broker treats the client as dead. In the browser using Javascript, on('message') fires by itself.

JSON in, JSON out

json.dumps() in Micropython, JSON.parse() in Javascript.

mqtt_dashboard.html is just a starting point

One HTML file, no backend, no build step. Extend it with charts, sliders, or anything else your project needs.

Lab 10 – Brokered Communication

